MLForge

Project Documentation

Universal Supervised Regression Training Platform

Version: 1.0

Stack: Python 3.10+ · Streamlit · scikit-learn · Optuna

Models: 19 algorithms across 7 groups

Export: Word (.docx) · PDF · LaTeX (.tex)

1. Project Overview

1.1 Purpose

MLForge is a general-purpose supervised regression training platform built on top of SVR implementations originally developed for the IndPenSim industrial penicillin fermentation simulation dataset (ML assignment). The project extends that work into a research-grade training environment where any tabular CSV or Excel dataset can be uploaded, preprocessed, trained across multiple models, compared, and exported as a formatted report.

A primary motivation is reproducibility: each paper in a hydrogen production ML literature review (papers_cleaned.csv, 29 papers) reports specific algorithms and R^2 values. MLForge is designed so that when a paper's dataset becomes available, those exact algorithms can be selected and run through the same pipeline, allowing direct comparison with published results.

1.2 Key Features

- Upload any tabular CSV or Excel dataset — no domain-specific hardcoding
- 19 regression models across 7 groups covering the full hydrogen/biofuel ML literature
- Configurable train/test split slider (50%–95%), CV folds, random seed
- Optuna-powered auto-tune with configurable trial budget (5–100 trials)
- Session comparison store — save multiple runs, compare with ΔR^2 highlighting
- One-click report export: Word (.docx), PDF, and LaTeX (.tex + figures zip)
- AI-style model interpretation with MAE/RMSE contextualisation and recommendations
- Dataset profiler with missing value detection, skewness check, time-series detection
- Literature paper matching via models_for_paper() helper function
- Hide slow models toggle for GP/Kernel Ridge on large datasets

1.3 Origin

The original assignment app (Penicillin Concentration Predictor) was a Streamlit multipage app with hardcoded SVR scratch implementations (LinearSVR_Scratch, PolySVR_Scratch, RBFSVRScratch), a fixed 80/20 split, and three benchmark models. MLForge retains the sklearn SVR kernels, removes the scratch implementations, and adds 13 new model families motivated by algorithm frequency analysis across 29 hydrogen production ML papers.

2. Project Structure

2.1 Folder Layout

File / Folder	Responsibility
Home.py	Main entry point. Dataset upload, profiling, session init.
requirements.txt	All Python dependencies with version pins.
Dockerfile	Container setup, exposes port 7860 for deployment.
assets/style.css	Global Streamlit CSS overrides. DM Sans font, metric cards.
config/__init__.py	Package marker.
config/sidebar_config.py	Persistent sidebar renderer. Returns cfg dict to all pages.
config/model_registry.py	Single source of truth for all 19 models.
config/param_spaces.py	Optuna/RandomizedSearchCV param spaces per model.
backend/__init__.py	Package marker.
backend/model_utils.py	Preprocessing, split, training, metrics, feature importance.
backend/autotune.py	Optuna search with RandomizedSearchCV and fixed-grid fallbacks.
backend/plotting.py	9 plot functions returning matplotlib Figures.
backend/analyze_helper.py	AI-style metric interpretation. Decoupled from Streamlit.
backend/session_store.py	Run history management in st.session_state.
backend/dataset_profiler.py	Dataset quality checks, skewness, missing values, time-series detection.
pages/1_Train_Diagnostics.py	Train single model, auto-tune, diagnostics, save run.
pages/2_Model_Comparison.py	Train multiple models on shared split, ranked table.
pages/3_Interactive_Prediction.py	Feature selection, reduced model, manual prediction ± RMSE.
pages/4_General_Testing.py	Test against any external dataset independently.
pages/5_Session_Comparison.py	View all saved runs, delta highlighting, visual comparison.
pages/6_Report_Generator.py	Configure and trigger report export in any format.
reports/__init__.py	Package marker.
reports/report_builder.py	Format-agnostic data assembly called by all exporters.
reports/docx_exporter.py	Word document builder using python-docx.
reports/pdf_exporter.py	HTML template via Jinja2 converted to PDF by WeasyPrint.
reports/latex_exporter.py	LaTeX .tex + figures bundled as .zip for Overleaf.

3. Architecture

3.1 Layer Overview

MLForge is structured as five functional layers. Each layer has a single responsibility and communicates with adjacent layers through well-defined interfaces (function returns, st.session_state keys).

Layer 1 — Data Ingestion

Home.py handles CSV/Excel upload, calls dataset_profiler.profile_dataset(), stores df, target_col, active_dataset, dataset_profile in st.session_state. All downstream pages read from session state — upload once, use everywhere.

Layer 2 — Preprocessing Controls

sidebar_config.render_sidebar() returns a cfg dict containing train_size, test_size, k_folds, split_strategy, random_seed, autotune_budget, hide_slow. Every page calls render_sidebar() first, passing cfg into all training calls. This replaces all hardcoded split values from the original app.

Layer 3 — Training Engine

backend/model_utils.py exposes train_model(), train_multiple_models(), and retrain_reduced(). Preprocessing (imputation, scaling, datetime parsing) is handled by preprocess() and shared across all training paths. backend/autotune.py runs Optuna TPE search with RandomizedSearchCV fallback.

Layer 4 — Results & Analysis

compute_metrics() returns MSE, RMSE, MAE, R^2. interpret_metrics() in analyze_helper.py produces structured analysis text decoupled from Streamlit (so the same text appears in both the UI and the exported report). 9 plot functions in plotting.py return matplotlib Figures for both display and embedding.

Layer 5 — Report Output

report_builder.py assembles a format-agnostic payload from session runs. docx_exporter, pdf_exporter, and latex_exporter each consume this payload independently, producing downloadable files via st.download_button.

3.2 Session State Keys

MLForge uses Streamlit session state as a lightweight in-process database. The following keys are written by Home.py and read by all pages:

Key	Contents
df	pd.DataFrame — the uploaded dataset
active_dataset	str — filename of the uploaded file
dataset_row_count	int — number of rows, shown in sidebar
dataset_col_count	int — number of columns, shown in sidebar
target_col	str — selected regression target column
dataset_profile	dict — output of profile_dataset(), used in reports
runs	list[dict] — saved training runs managed by session_store.py

3.3 Data Flow — Single Training Run

1. User uploads CSV on Home.py → df stored in session state
2. User selects target column → target_col stored in session state
3. User navigates to Page 1, configures model and sidebar settings
4. Train button → train_model(df, target_col, model_key, cfg) called
5. preprocess() → split_data() → model.fit() → model.predict() → compute_metrics()
6. Results dict returned: {model, metrics, data, scaler, imputer, feature_names}
7. Plots rendered via plotting.py, analysis via interpret_metrics()
8. User clicks Save → save_run() appends to st.session_state['runs']
9. User navigates to Page 6 → build_report_payload() assembles data → exporter generates file